



AN IMS [ISO 21001, ISO 9001, ISO 14001, ISO 45001] CERTIFIED TRAINING INSTITUTION

NETTUR TECHNICAL TRAINING FOUNDATION

PROJECT REPORT

ON

CODE TO ARCHITECTURE VISUALIZER

PROJECT WORK DONE BY

RANJAN BABU V	NEC0823004
SHWETHA A	NEC0823029
AKLIMA BEGUM SARA	NEC0823115
GOWTHAM S	NEC0823105
YOGESH S	NEC0823014

DIPLOMA IN
COMPUTER ENGINEERING & IT INFRASTRUCTURE
NTTF ELECTRONICS CENTER
ELECTRONICS CITY, BANGALORE – 560100
2023 – 2026



AN IMS [ISO 21001, ISO 9001, ISO 14001, ISO 45001] CERTIFIED TRAINING INSTITUTION

NETTUR TECHNICAL TRAINING FOUNDATION

CERTIFICATE

This is to certify that the project titled

CODE TO ARCHITECTURE VISUALIZER

is a Bonafede report of project work done by

RANJAN BABU V	NEC0823004
SHWETHA A	NEC0823029
AKLIMA BEGUM SARA	NEC0823115
GOWTHAM S	NEC0823105
YOGESH S	NEC0823014

IN PARTIAL FULFILLMENT OF THE REQUIREMENT FOR THE AWARD OF DIPLOMA IN
COMPUTER ENGINEERING & IT INFRASTRUCTURE UNDER THE INSTITUTION NETTUR
TECHNICAL TRAINING FOUNDATION, ELECTRONICS CENTRE, BANGALORE DURING THE
ACADEMIC YEAR 2023-2026

PROJECT GUIDE

PROJECT COORDINATOR

COURSE HEAD

INTERNAL EXAMINER

EXTERNAL EXAMINER

PRINCIPAL

ACKNOWLEDGEMENT

We would like to express our deepest gratitude and appreciation to all those who have supported us throughout the completion of this final year project.

To begin with, we would like to thank Principal **Dr. Somanathan K, NTTF NEC**, for his consistent encouragement and support which pushed us to do better. We are thankful to our Course Head of CP08 **Mr. Loganathan B** and Program Manager **Mrs. Preetha S** for being a source of inspiration throughout our project.

We would also like to extend our sincere thanks to our project coordinators **Mrs. Preetha S**, for their invaluable guidance, constant encouragement, and constructive feedback. Their expertise and mentorship have played a crucial role in shaping this project and enhancing its quality. We are indebted and grateful to **Mr. Loganathan B** our project guide for his valuable guidance for each and every stage of our project as reality.

In conclusion, this project would not have been possible without the collective efforts and support of all those mentioned above and the team members of the project. We are truly grateful for their contributions, and we feel fortunate to have been surrounded by such incredible individuals during this project.

ABSTRACT

The **Code-to-Visualizer System** is an intelligent web-based application designed to simplify code understanding by converting source code into visual flowcharts and structured explanations. The system allows users to write or upload Python and JavaScript programs, execute them securely, and automatically generate flowchart representations that illustrate the program logic in an easy-to-understand manner. This helps students, beginners, developers, and educators analyze complex code structures more efficiently.

The system includes secure user authentication with OTP verification, encrypted code storage, password hashing, and admin monitoring features to ensure data security and controlled access. Additional functionalities such as code execution, downloadable flowcharts, recent activity restoration, task-based learning modules, and performance tracking improve the overall learning and development experience. The application also provides an admin dashboard for monitoring users, submissions, reports, and system activities.

The main objective of the project is to reduce the difficulty of understanding programming logic by transforming textual source code into interactive visual representations. By integrating code execution, visualization, and security features into a single platform, the system enhances programming education, debugging, and logical analysis in an efficient and user-friendly manner.

INDEX

SLNO.	CONTENT	PAGE NO
CHAPTER - I INTRODUCTION		
1.1	Introduction.....	1
1.2	Objective of the Project.....	3
1.3	System Study.....	4
1.3.1	Study on Existing System.....	4
1.3.2	Demerits of Existing System.....	5
1.3.3	Study on Proposed System.....	5
1.3.4	Merits of Proposed System.....	6
1.3.5	Literature Survey	7
CHAPTER – II SYSTEM ANALYSIS		
2.1	User Requirement Specifications.....	8
2.2	Software Requirement Specification (SRS).....	9
2.3	System Specifications.....	11
2.3.1	Hardware Specifications.....	11
2.3.2	Software Specifications.....	12
2.4	Final Outline of Proposed System.....	13
CHAPTER – III DESIGN AND DEVELOPMENT PROCESS		
3.1	Fundamental Design Concepts.....	14
3.1.1	Modules and Functionalities.....	15
3.1.2	Description of Modules	18
3.2	Design Notations.....	20
3.2.1	DFD Symbols.....	21
3.2.2	Data Flow Diagram.....	21
3.2.3	ER Diagram.....	23
3.3	Design Process.....	24
3.3.1	Input Design.....	24

3.3.2	Output Design.....	26
3.3.3	Database Design.....	26
3.4	Development Approach.....	29
3.4.1	Development Model / Approach.....	29
CHAPTER – IV PLANNING		
4.1	Gantt Chart.....	30
CHAPTER – V TESTING AND IMPLEMENTATION		
5.1	Testing.....	32
5.1.1	Testing and Methodologies.....	33
5.1.2	Test Cases.....	34
5.2	System Implementation.....	40
5.2.1	Implementation Procedures.....	40
CHAPTER – VI ADVANTAGES AND DISADVANTAGES		
6.1	Advantages and Disadvantages	
CHAPTER – VII CONCLUSION		
7.1	Conclusion.....	44
7.2	Scope for Future Enhancements.....	46
ANNEXURE - A		
	Sample Code.....	47
ANNEXURE - B		
	Abbreviation	53
	Bibliography	54

CHAPTER – I INTRODUCTION

1.1 INTRODUCTION

In today's software-driven world, understanding how a program flows and behaves is a fundamental challenge faced by students, developers, and educators alike. Reading raw source code — especially for beginners — is a cognitively demanding task that often leads to confusion, errors, and slow learning. Traditional debugging tools and textbooks provide limited visual context, leaving learners to mentally simulate program execution on their own.

Ragsy (Code Architecture Visualizer) is a full-stack web application designed to bridge this gap. It transforms Python and JavaScript source code into interactive, step-by-step flowcharts using Abstract Syntax Tree (AST) analysis and graph-based rendering. Users paste their code into a browser-based editor, and the platform instantly produces a structured, colour-coded flowchart that visually represents the control flow, function calls, loops, conditionals, and return statements of the program.

Beyond flowchart generation, Ragsy provides a live code execution environment where users can run their code directly in the browser, observe real-time output, and provide input values interactively. The platform supports both Python Basics (executed via a sandboxed server-side Python runtime) and JavaScript Basics (executed via Node.js). All code submissions are securely stored in an AES-256 encrypted database, and users can retrieve their recent activity to restore and re-analyse past work. Ragsy is built with a modern technology stack: React (Vite) on the frontend, Fast API (Python) on the backend, and Microsoft SQL Server as the development database (PostgreSQL for production deployment).

1.2 OBJECTIVE OF OUR PROJECT

1. To provide an interactive visual representation of Python and JavaScript source code as flowcharts, helping learners and developers understand program logic without deep manual analysis.
2. To offer a live, sandboxed code execution environment supporting both Python and JavaScript directly in the browser.
3. To implement a secure, multi-user authentication system with SHA-256 hashed passwords, SMS-based OTP password recovery via Fast2SMS, and 24-hour session expiry.
4. To encrypt all user-submitted code in the database using AES-256 (Fernet), ensuring complete user data privacy and platform trustworthiness.
5. To build an Admin Dashboard that allows platform administrators to monitor users, view submission statistics, and manage the system without direct database access.
6. To support high-quality downloadable flowchart exports as PNG images, capturing all node connections and SVG edges using html-to-image.
7. To create a clean, responsive, and humanised user interface accessible to students and beginner programmers without any installation.
8. To design the system for free cloud deployment using open-source tools (Vercel, Railway, Supabase, Fast2SMS).

1.3 SYSTEM STUDY

1.3.1 STUDY ON EXISTING SYSTEM

Several tools currently exist for visualising code or understanding program flow:

1. **Python Tutor (pythontutor.com)**: Animates Python code execution step by step. Useful for teaching but does not produce flowcharts and supports Python only.
2. **VS Code Extensions**: IDE-based plugins generating basic flowcharts. Require installing a development environment — inaccessible to beginners.
3. **Lucid chart and Draw.io**: Manual diagramming tools where users hand-draw flowcharts. Time-consuming and require prior knowledge of flowchart conventions.
4. **Code2Flow**: Generates flowcharts from pseudocode, not actual source code. Limited language support and no interactive execution environment.
5. **Mermaid.js**: A text-based diagram tool requiring users to write diagram syntax, not actual code. Not suitable for direct code visualisation.

1.3.1 DEMERITS OF EXISTING SYSTEM

1. No existing platform combines live code execution with real-time flowchart generation in a single interface.
2. Most tools are desktop-based or require installation, making them inaccessible to students without technical setup knowledge.
3. Existing tools do not support both Python and JavaScript in the same platform.
4. None of the existing tools offer encrypted code storage, multi-user accounts, SMS-based password recovery, or an Admin Dashboard.
5. Flowchart export quality in existing tools is often poor, with edges and connections not properly captured in exported images.
6. No existing tool is designed specifically for beginner programmers learning basic programming concepts interactively.

1.3.2 STUDY ON PROPOSED SYSTEM

Ragsy addresses the shortcomings of existing systems by providing a unified, web-based platform that combines code visualisation, execution, and secure data management. The proposed system is fully browser-based (no installation required), language-aware (Python Basics and JavaScript Basics), secure by design (hashed passwords, encrypted submissions, session expiry), and admin-controlled (dedicated dashboard for platform oversight).

1.3.3 MERITS OF PROPOSED SYSTEM

1. Real-time flowchart generation from actual source code using AST analysis, with Dager automatic layout ensuring non-overlapping node placement.
2. Live code execution with input/output support directly in the browser for both Python and JavaScript.
3. High-quality PNG export of flowcharts including all edge connections and arrows, rendered using html-to-image.
4. Secure multi-user authentication with SHA-256 hashed passwords and Fast2SMS SMS OTP-based password recovery.
5. AES-256 (Fernet) encryption of all code submissions stored in the database — only the submitting user can retrieve their code.
6. Admin Dashboard with real-time platform statistics, user management (view/delete), and encrypted submission monitoring.
7. Recent Activity panel allowing users to restore and re-analyse previously submitted code.
8. Humanised, responsive UI built with React for an approachable, accessible learning experience.
9. Designed for zero-cost cloud deployment: Vercel (frontend), Railway (backend), Supabase (PostgreSQL), Fast2SMS (SMS OTP).

1.3.4 LITERATURE SURVEY

The Python Software Foundation Python 3.12 documentation for the AST (Abstract Syntax Tree) module provided the foundation for AST-based code parsing and the generation of flowchart nodes and edges in the Ragsy application. The React Flow Documentation was used as the primary reference for implementing interactive flowchart rendering features such as node dragging, zooming, and graphical visualization. The Dager Graph Layout Algorithm supported the automatic hierarchical arrangement of flowchart nodes to avoid overlapping during rendering.

The OWASP Password Storage Cheat Sheet guided the implementation of SHA-256 password hashing and session security design within the system. The Fast2SMS Phone Authentication Documentation served as the basis for implementing SMS OTP-based password recovery functionality. Additionally, FastAPI Documentation was referenced for backend development due to its asynchronous performance, automatic OpenAPI documentation support, and built-in validation features.

CHAPTER – II

SYSTEM ANALYSIS

2.1 USER REQUIREMENT SPECIFICATIONS (URS)

The system shall allow users to register using a username, mobile number, and password. Registered users shall be able to log in securely and access a personalised dashboard. The application shall enable users to paste Python or JavaScript code and instantly generate a corresponding flowchart. Users shall also be able to execute code and view real-time standard output and error messages. When the program requires user input, the system shall provide an interactive method for entering input values during execution.

Additionally, the application shall allow users to download generated flowcharts as high-quality PNG images with connected edges. Users shall be able to view, restore, and re-analyse their recent code submissions for future reference. The system shall support password reset functionality through SMS OTP verification sent to the user's registered mobile number. Admin users shall have access to a separate dashboard containing user information and submission details. Furthermore, the system shall ensure that user credentials, submitted code, and other sensitive data are encrypted and securely protected to maintain privacy and security.

2.2 SOFTWARE REQUIREMENT SPECIFICATION (SRS)

FUNCTIONAL REQUIREMENTS

1. FR-01: The system shall parse Python code using the ast module and JavaScript using a custom tokenizer to generate flowchart nodes and edges.

2. FR-03: The system shall hash all passwords using SHA-256 before storage. Plain text passwords shall never be stored.
3. FR-04: The system shall encrypt all code submissions using AES-256 (Fernet) before storage, and decrypt them only for the submitting user.
4. FR-05: The system shall send a 6-digit SMS OTP via Fast2SMS Phone Auth for password reset, expiring after 5 minutes.
5. FR-06: The system shall automatically expire user sessions after 24 hours.
6. FR-07: The system shall apply Dagre auto-layout to prevent node overlapping on initial flowchart render.
7. FR-08: The Admin Dashboard shall display total users, submissions, flowcharts generated, and new users per week.
8. FR-09: Admin users shall be able to delete non-admin user accounts from the dashboard.
9. FR-10: The system shall export flowcharts as PNG images capturing all SVG edges using html-to-image.

NON-FUNCTIONAL REQUIREMENTS

1. NFR-01 Performance: Code execution shall complete within 5 seconds; flowchart generation within 3 seconds for programs up to 500 lines.
2. NFR-02 Security: No plain-text passwords or unencrypted code shall ever be stored in the database.

3. NFR-03 Usability: The platform shall be fully functional on modern browsers (Chrome, Firefox, Edge) without any client-side installation.
4. NFR-04 Scalability: The API shall be stateless and deployable on containerized infrastructure (Railway, Render, etc.).
5. NFR-05 Availability: The deployed system shall target 99% uptime using cloud-managed services.

2.3 SYSTEM SPECIFICATIONS

2.3.1 HARDWARE SPECIFICATIONS

Components	Minimum Requirements
Processor	: Intel Core i3 / AMD Ryzen 3
RAM	: 4 GB
Storage	: 256 GB HDD
Network	: Broadband 1 Mbps
Display	: 1280 × 720

2.3.2 SOFTWARE SPECIFICATIONS

CATEGORY	TECHNOLOGY
Frontend Framework	: React via Vite
Backend Framework	: FastAPI (Python)
Database (Dev)	: Microsoft SQL Server
Database (Prod)	: PostgreSQL (Supabase)
JS Execution Runtime	: Node.js
Flow Rendering	: React Flow (@xyflow/react)
Graph Auto-Layout	: Dagre
PNG Export	: html-to-image
SMS OTP	: Fast2SMS Phone Auth
Code Encryption	: cryptography (Fernet/AES-256)
Password Hashing	: hashlib SHA-256
Deployment (FE)	: Vercel
Version Control	: Git / GitHub

2.4 FINAL OUTLINE OF PROPOSED SYSTEM

Ragsy is structured as a three-tier web application. The Presentation Tier is a React SPA (Vite) served from Vercel, handling all UI rendering, flowchart display via React Flow, and code editing. The Application Tier is a FastAPI REST API served from Railway, handling authentication, AST parsing, sandboxed execution, database operations, and admin functions. The Data Tier is Microsoft SQL Server in development and PostgreSQL (Superbase) in production, storing encrypted user data, code submissions, flowchart metadata, OTP records, and activity logs. All communication between tiers uses JSON over HTTPS, and no sensitive data is transmitted or stored in plain text.

The system is designed with scalability, maintainability, and security as its primary objectives. Each tier operates independently, allowing updates and enhancements without affecting other components of the application. Secure authentication mechanisms, encrypted data storage, and role-based access control help protect sensitive user information. The architecture also supports efficient processing of code execution, flowchart generation, and AI-powered explanations. The system is designed with scalability, maintainability, and security as its primary objectives. Each tier operates independently, allowing updates and enhancements without affecting other components of the application. Secure authentication mechanisms, encrypted data storage, and role-based access control help protect sensitive user information. The architecture also supports efficient processing of code execution, flowchart generation, and AI-powered explanations.

CHAPTER – III

DESIGN AND DEVELOPMENT PROCESS

3.1 FUNDAMENTAL DESIGN CONCEPTS

3.1.1 MODULES AND FUNCTIONALITIES

Authentication Module

Handles all user identity operations. On registration, passwords are SHA-256 hashed before storage. On login, a unique 32-character alphanumeric token is generated and stored in an in-memory session dictionary with a 24-hour expiry timestamp. The forgot password flow invokes Fast2SMS Phone Authentication to deliver an SMS OTP to the user's registered mobile number. Admin users (is_admin=1 in USERS table) are automatically routed to the Admin Dashboard on login

Dashboard Module

The Dashboard Module serves as the main entry point of the Ragsy application after a user logs in successfully. It provides a personalized greeting to welcome the user and enhance the user experience. The dashboard allows users to choose between Python Basics and JavaScript Basics learning paths based on their interests. Each language option is displayed as a separate card for easy navigation and accessibility. When a user selects a language, the system stores the choice and redirects them to the code editor page. The selected language is automatically configured in the editor, allowing users to start coding immediately. The dashboard also acts as a central navigation hub for accessing features such as tasks, recent activities, and flowchart generation

Code Editor Module

The Code Editor Module is the core component of the Ragsy application where users write and execute their programs. It provides a dedicated text area for entering Python or JavaScript code in an organized manner. The module includes a sample code loader that helps beginners quickly understand the coding format and syntax. A language indicator clearly displays the currently selected programming language to avoid confusion. Users can edit, modify, and test their code directly within the editor interface. The **Run Code** trigger allows users to execute their programs and view the output instantly. The **Generate Flowchart** trigger converts the source code into a visual flowchart representation for better understanding. Overall, the Code Editor Module offers an interactive environment for coding, execution, and program visualization.

FLOWCHART ENGINE

The core feature of Ragsy. The backend parses submitted code using Python's built-in ast module (Python) or a custom tokenizer (JavaScript), walking the syntax tree to identify control flow constructs: function definitions, if/elif/else branches, for/while loops, try/except blocks, return statements, and function calls. Each construct becomes a typed node (process, decision, terminal, I/O) in a directed graph; relationships become labelled edges. The backend returns nodes and edges as JSON. On the frontend, the Dagre library recalculates positions using a top-to-bottom hierarchical layout (TB direction, nodesep=60, ranksep=80) before React Flow renders the interactive diagram.

EXECUTION ENGINE

Runs user code in an isolated environment. Python code executes in a subprocess with stdout/stderr capture and a strict 5-second timeout to prevent infinite loops. Before execution, the code is scanned for input() calls; if detected, a modal prompts the user to provide values which are injected as simulated stdin. JavaScript code executes via a Node.js child process with an injected input() shim for interactive programs. The subprocess design mitigates dangerous operations by isolating execution from the server's file system and network.

Explanation Module

The Explanation Module helps users understand their programs by generating simple and easy-to-read explanations of the submitted code. It uses AI technology to analyze the code structure, syntax, and program logic automatically. The module converts complex programming statements into plain English descriptions that are easier for beginners to understand. It explains the purpose of variables, conditions, loops, functions, and other programming elements used in the code. Users can quickly learn how their program works without manually analyzing each line. The generated explanations improve learning efficiency and help users identify coding mistakes more easily. The module is especially useful for students who are new to programming concepts and flow control. Overall, it enhances code comprehension by providing clear and user-friendly explanations of program behavior.

Recent Activity Module

The Recent Activity Module allows users to view their previously submitted programs and related activities. It maintains a history of code submissions along with details such as language type and submission time. Users can easily restore any previous code submission back into the editor with a single click. When code is restored, the correct programming language is automatically selected to ensure proper execution. The module also records important user actions, creating an activity log for tracking and reference purposes. Overall, it improves productivity by enabling quick access to past work and maintaining a record of user activities.

3.1.2 DESCRIPTION OF FUNCTIONALITIES

Authentication Module

The Authentication Module manages user registration, login, logout, and secure session handling within the application. It provides password recovery through SMS OTP verification and identifies admin users for role-based access control.

Dashboard Module

The Dashboard Module allows users to select either Python Basics or JavaScript Basics based on their learning preference. It displays a personalized greeting after login and provides a user-friendly interface. The module also enables seamless navigation to the code editor for coding and learning activities.

Code Editor Module

The Code Editor Module provides a text area where users can write and edit Python or JavaScript code. It includes a sample code loader and a language indicator to assist users while coding.

Flowchart Engine

The Flowchart Engine Module converts source code into a visual flowchart using AST parsing and JSON-based node and edge generation. It applies the Dagre layout algorithm and renders diagrams using React Flow with interactive drag and zoom features

Execution Engine

The Code Execution Module runs Python programs in a sandboxed subprocess and JavaScript programs using a Node.js child process. It supports user input through an input scanning modal and captures both standard output (stdout) and error output (stderr).

Explanation Module

The Explanation Module uses AI technology to analyze the submitted source code automatically. It converts program structure and logic into simple, plain-English explanations that are easy to understand.

Recent Activity Module

The Recent Activity Module displays a history of all code submissions made by the user. It allows users to review their previously executed programs and results. The code restore feature reloads saved code into the editor with the correct programming language selected automatically.

3.2 DESIGN NOTATIONS

3.2.1 DFD SYMBOLS

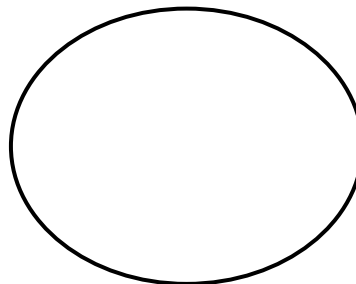
A Square defines the source/destination of system data.



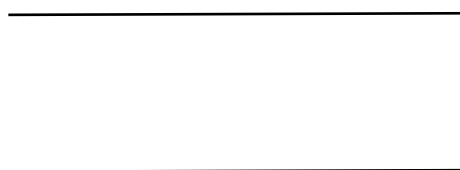
An arrow identifies data flow. It is the pipeline through which information flows.



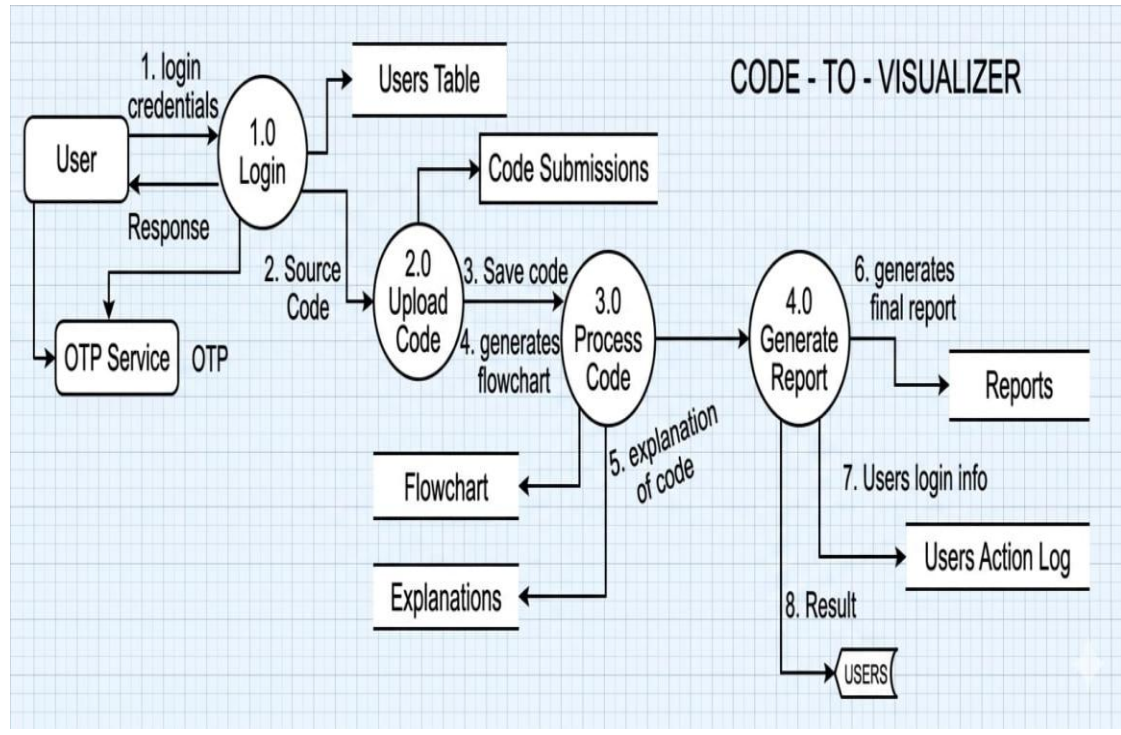
circle represents a process that transforms incoming dataflow into outgoing dataflows



Open rectangle represents the data store or tab.



3.2.2 DFD FLOW DIAGRAM



The Data Flow Diagram (DFD) of the Code-to-Visualizer System illustrates the flow of data between users, processes, and data stores within the system.

1. Login(1.0):

The process begins when the User enters login credentials into the system. The credentials are verified using the Users Table. An OTP Service is used to send and validate the OTP for secure authentication. After successful verification, a response is returned to the user.

2. Upload_Code(2.0):

After logging in, the user uploads the source code to the system. The uploaded code is stored in the Code Submissions data store for future reference and processing.

3. **Process_Code(3.0):**

The uploaded code is analyzed and processed by the system. During this stage, the system saves the code, generates a flowchart representation, and produces a detailed code explanation. The generated outputs are stored in the Flowchart and Explanations data stores.

4. **Generate_Report(4.0):**

The processed results are then sent to the report generation module. This module creates the final report by combining the generated flowchart, code explanations, and processing details. The report is stored in the Reports data store.

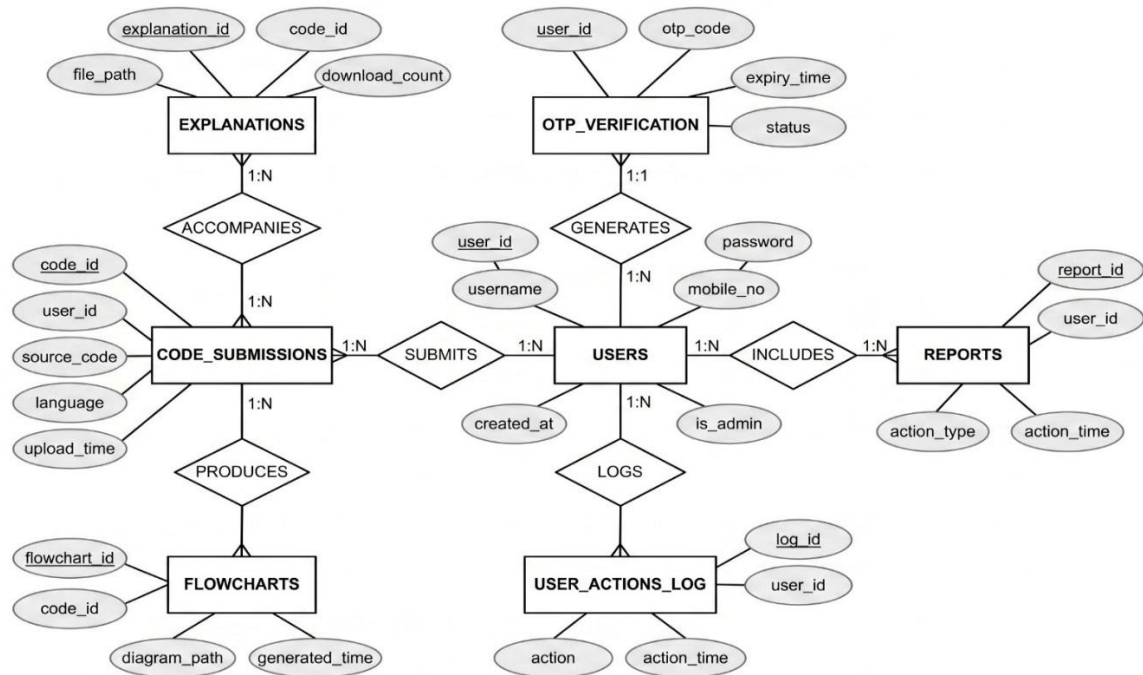
5. **User_Activity_Logging:**

The system records user login information and actions in the Users Action Log for monitoring, auditing, and security purposes.

6. **Result_Delivery:**

Finally, the generated report and processed results are delivered back to the Users, allowing them to view the visualized code output, explanations, and reports.

3.2.3 ER DIAGRAM



The **ER Diagram** of the Ragsy Code Architecture Visualizer system shows the database structure and relationships between different entities. The **USERS** entity stores user details such as username, password, mobile number, and admin status. Users submit programs through the **CODE_SUBMISSIONS** entity, which stores source code, programming language, and upload information. Each code submission can generate **FLOWCHARTS** and **EXPLANATIONS** to help users understand program logic visually. The **OTP_VERIFICATION** entity provides secure authentication and password recovery using OTP codes. Additional entities such as **REPORTS** and **USER_ACTIONS_LOG** track user activities and maintain system records for monitoring and management purposes.

3.3 DESIGN PROCESS

3.3.1 INPUT DESIGN

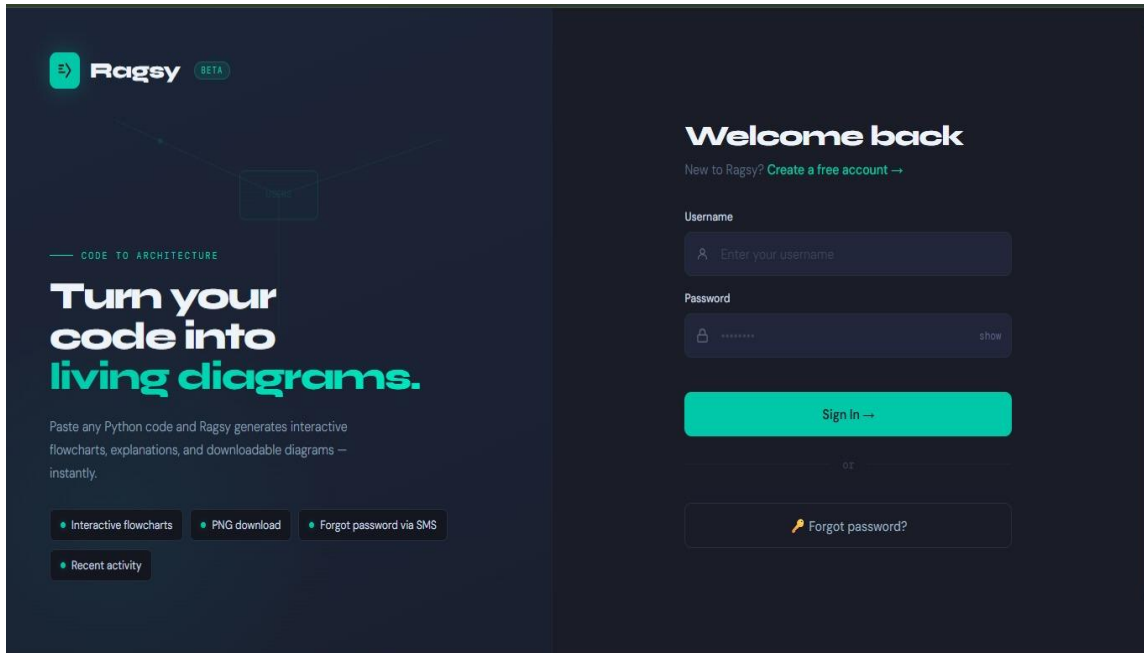


Figure.3.3.1 INPUT DESIGN

The Ragsy User Login Page features a modern dark-themed interface designed for secure and easy user access. The left section introduces the platform and highlights its ability to convert source code into interactive flowcharts and architecture diagrams. It also showcases features such as flowchart generation, PNG downloads, recent activity tracking, and SMS-based password recovery. The right section contains the login form where users can enter their username and password to sign in or create a new account. A Forgot Password option allows users to recover their accounts using OTP (One-Time Password) verification sent to their registered mobile number. This OTP-based authentication enhances security by ensuring that only authorized users can reset their passwords and access their accounts.

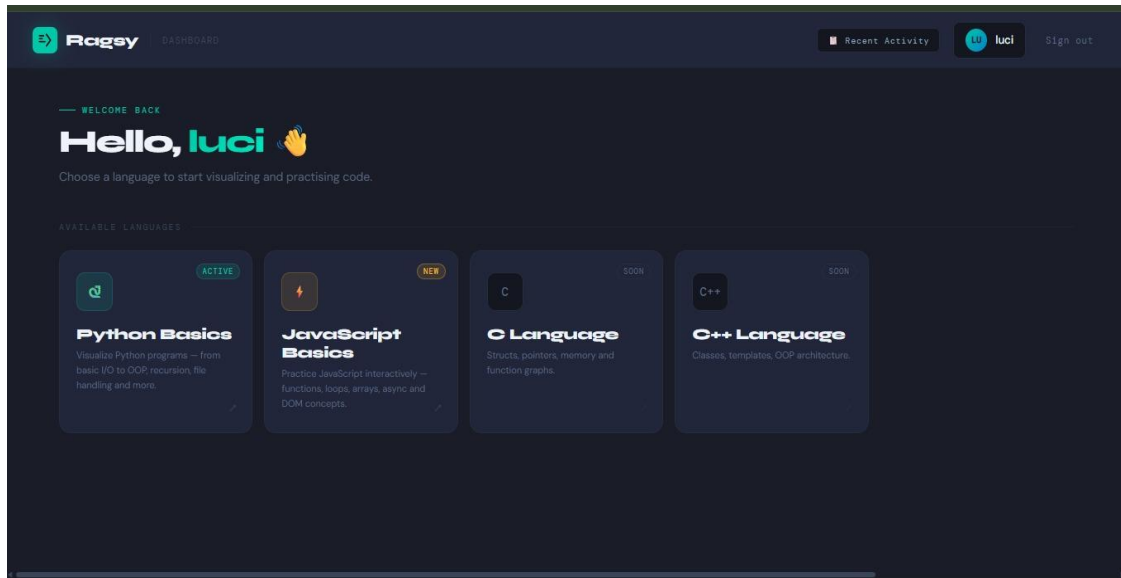


Figure.3.3.1 INPUT DESIGN

The Ragsy Dashboard Page serves as the main workspace for users after logging into the system. It features a modern dark-themed interface with easy navigation, including the Ragsy logo, Recent Activity option, user profile, and Sign Out button. The dashboard welcomes the user with a personalized greeting and allows them to select a programming language for code visualization and practice. Available modules include Python Basics and JavaScript Basics, covering important programming concepts and exercises. Support for C and C++ languages is also displayed as upcoming features. Overall, the dashboard acts as a central hub for accessing learning modules, managing activities, and navigating the Ragsy platform efficiently.

3.3.2 DATABASE DIAGRAM

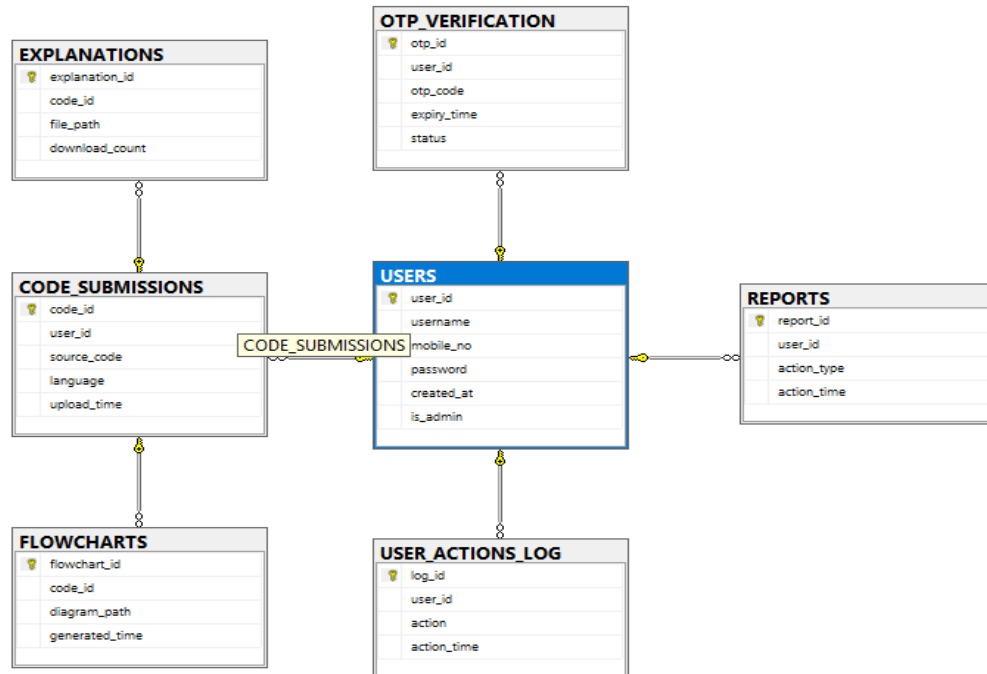


Figure.3.3.2 database diagram

The Entity Relationship Diagram (ERD) of the Ragsy system consists of seven main tables: **USERS**, **CODE_SUBMISSIONS**, **FLOWCHARTS**, **EXPLANATIONS**, **OTP_VERIFICATION**, **USER_ACTIONS_LOG**, and **REPORTS**. The **USERS** table acts as the central table and stores user details such as username, mobile number, password, account creation date, and admin status. The **CODE_SUBMISSIONS** table stores the source code uploaded by users and is linked to the **USERS** table through the `user_id` field. Generated flowcharts are stored in the **FLOWCHARTS** table, while explanation files related to submitted code are maintained in the **EXPLANATIONS** table. The **OTP_VERIFICATION** table manages OTP-based password recovery and user verification processes. User activities such as login, code execution, and other actions are recorded in the **USER_ACTIONS_LOG** table.

3.3.3 OUTPUT DESIGN

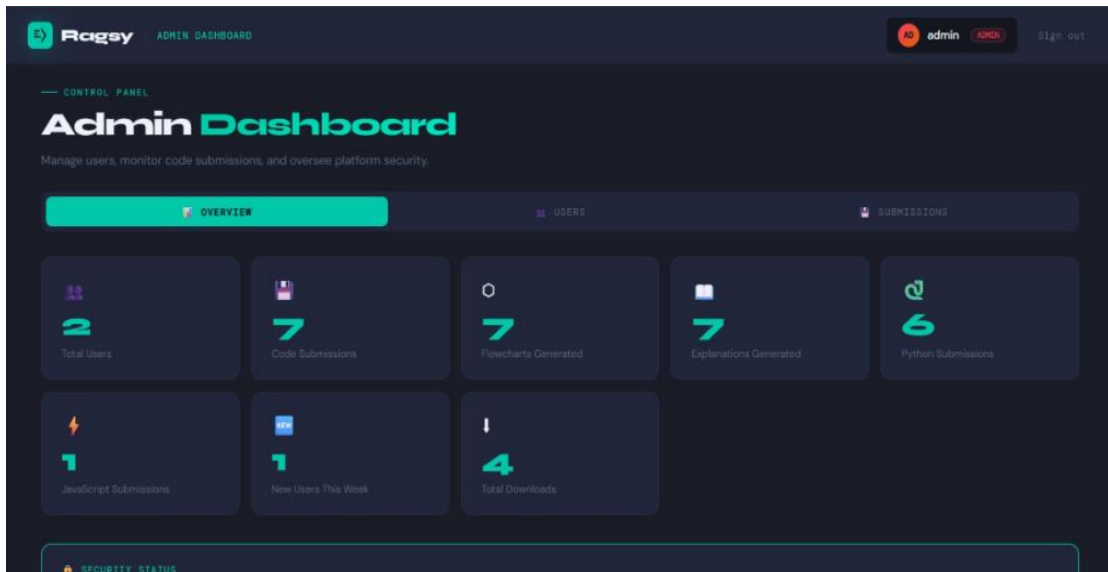


Fig.3.3.3 admin dashboard

The image displays the Admin Dashboard Page of the Ragsy Code Architecture Visualizer system, which provides administrators with a centralized interface for monitoring and managing platform activities. The dashboard features a modern dark-themed design with easy navigation and an organized layout. At the top, administrators can access options such as the admin profile and sign-out functionality. The dashboard presents key statistics, including the total number of users, code submissions, flowcharts generated, explanations generated, and programming language submissions. It also displays information about new users registered during the current week and the total number of downloads performed on the platform. Separate sections such as Overview, Users, and Submissions allow administrators to efficiently manage user accounts and monitor system usage. Overall, the Admin Dashboard serves as a powerful control panel for tracking platform performance, overseeing user activities, and maintaining system security and administration.

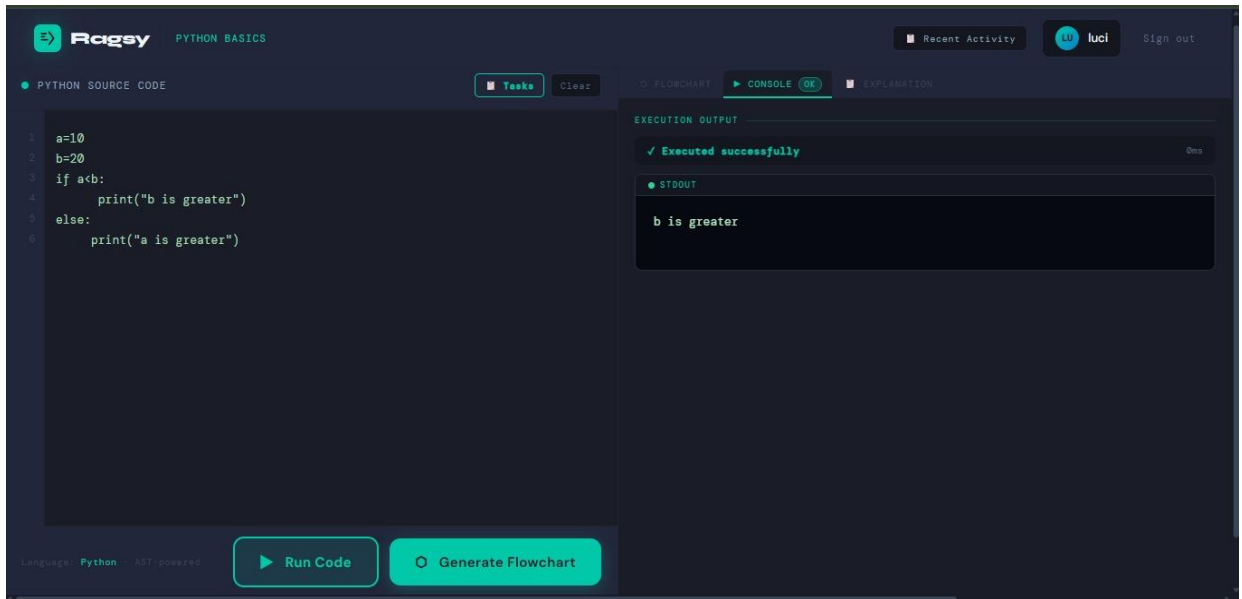


Fig.3.3.2 OUTPUT DESIGN

The image displays the **Python Code Editor and Execution Page** of the Raggy Code Architecture Visualizer system. It provides an interactive workspace where users can write, execute, and analyze Python programs. The left side contains a code editor with a sample Python program that uses an **if-else** statement to compare two variables. Users can easily enter or modify code and execute it using the **Run Code** button provided at the bottom. The **Generate Flowchart** button allows the source code to be converted into a visual flowchart representation. On the right side, the **Console** panel displays the execution status and output of the program. In this example, the output shows "**b is greater**", indicating successful execution of the code. Overall, this page helps users understand program logic by combining code execution, output visualization, and automated flowchart generation in a single environment.

3.4 DEVELOPMENT APPROACH

3.4.1 DEVELOPMENT MODEL / APPROACH

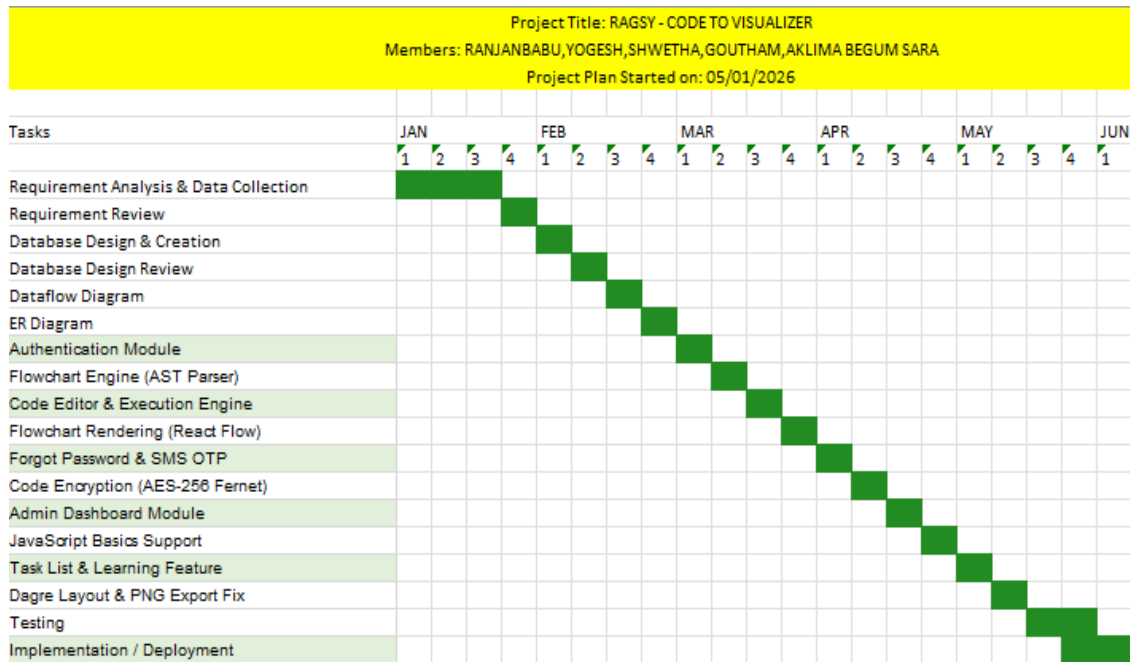
The development of the Ragsy application was carried out in four sprints over an eight-week period. During ****Sprint 1 (Week 1–2)****, the focus was on establishing the core infrastructure of the system. This included setting up the FastAPI backend framework, designing the SQL Server database schema, implementing user registration and login APIs, and developing a React Single Page Application (SPA) with routing and basic authentication interfaces. In ****Sprint 2 (Week 3–4)****, the team concentrated on implementing the core features of the application, such as the AST parser, flowchart generation engine, React Flow-based visualization, code execution engine, and console output panel for displaying program results.

During ****Sprint 3 (Week 5–6)****, emphasis was placed on security and user experience improvements. Features such as SHA-256 password hashing, Fernet-based code encryption, Dagre layout integration for improved flowchart positioning, PNG export functionality, Firebase OTP-based forgot password support, and UI enhancements were implemented. Finally, ****Sprint 4 (Week 7–8)**** focused on administration and deployment activities, including the development of the Admin Dashboard, JavaScript Basics learning module, Recent Activity tracking, session expiry management, cloud deployment, and preparation of project documentation. This phased sprint-based approach ensured systematic development, testing, and delivery of the application

CHAPTER – IV

PLANNING

4.1 GANTT CHART



A Gantt chart is a project management tool used to visually represent the schedule and progress of a project over a specific period of time. It displays the various tasks or activities involved in the project along with their start dates, end dates, and duration using horizontal bars. The chart provides a clear overview of the project timeline, making it easier to understand the flow of activities and the status of ongoing tasks.

The chart helps in planning, organizing, and tracking project activities efficiently by showing the sequence of tasks and their dependencies. It is widely used in software development and final year projects to monitor progress, manage deadlines, allocate resources, and ensure the timely completion of the project. By using a Gantt chart, project teams can improve coordination, identify delays early, and manage work more effectively.

CHAPTER – V

TESTING AND IMPLEMENTATION

5.1 TESTING

5.1.1 TESTING AND METHODOLOGIES

Unit Testing:

Individual backend functions including `hash_pw()`, `encrypt code()`, `decrypt code()`, `gen_otp()`, `run_code_safe()`, and `db_user_create()` were tested in isolation to verify correct behaviour, error handling, and edge cases.

Integration Testing

All API endpoints were tested using the FastAPI TestClient, verifying correct HTTP status codes, response schemas, and database interactions for routes including `/auth/register`, `/auth/login`, `/auth/forgot-password`, `/auth/reset-password`, `/visualize`, `/run`, `/activities`, and all `/admin/*` routes.

Frontend Testing

React components(AUTHPage, ForgotPasswordView, FlowDiagram, Editor Page, AdminPage) were tested manually in the browser for state management, rendering correctness, error display, and complete user interaction flows.

Security Testing

Password hashing was verified by confirming SHA-256 output consistency. Fernet encryption/decryption round-trips were tested to ensure code retrieval integrity. Admin route protection was confirmed by verifying HTTP 403 responses for non-admin tokens.

5.1.2 Test Cases

TC ID	Description	Input	Expected Result	Actual Result	Result
TC-01	User Registration – Valid Input	Username: ranjan, Mobile: 9786006541, Password: Test@123	Account created successfully. Token returned. Redirected to Dashboard.	Account created successfully. Token returned. Redirected to Dashboard.	PASS
TC-02	User Registration – Duplicate Username	Username: ranjan (already exists), Mobile: 9786006542	HTTP 400 error: Username already taken	Second account created with same username.	FAIL
TC-03	User Registration – Duplicate Mobile Number	Username: newuser, Mobile: 9786006541	HTTP 400 error: Mobile already registered	HTTP 400 error shown.	PASS
TC-04	User Registration – Empty Fields	Empty Username, Mobile, Password	Validation errors displayed	HTTP 500 Internal Server Error	FAIL
TC-05	User Login – Valid Credentials	Username: ranjan, Password: Test@123	Login successful	Login successful	PASS
TC-06	User Login – Wrong Password	Username: ranjan, Password: WrongPass	HTTP 401 Incorrect Password	Login succeeded	FAIL
TC-07	User Login – Non-existent Username	Username: unknownuser	HTTP 401 Username not found	HTTP 500 returned	FAIL
TC-08	Admin Login – Valid Credentials	Username: admin, Password: admin	Redirect to Admin Dashboard	Redirect to Admin Dashboard	PASS
TC-09	Regular User Cannot Access Admin	Username: ranjan	Redirect to User Dashboard	Redirected to Admin Dashboard	FAIL
TC-10	Session Expiry After 24 Hours	Expired Token	HTTP 401 Session Expired	Token accepted	FAIL
TC-11	Forgot Password – Registered Mobile	Registered mobile number	OTP sent	OTP sent	PASS

TC-12	Forgot Password — Unregistered Mobile	Mobile: +919000000000 (not registered)	HTTP 404 error: No account found with this mobile number	Application crashed with unhandled exception. No user-friendly error shown.	FAIL
TC-13	Forgot Password — Correct OTP Entered	Correct 6-digit OTP entered within 5 minutes	OTP verified. Step 3 (New Password) displayed.	OTP verified message shown, but page did not advance to Step 3. UI state update missing.	FAIL
TC-14	Forgot Password — Wrong OTP Entered	Incorrect 6-digit OTP	HTTP 400 error: Incorrect OTP. Please try again.	HTTP 400 error: Incorrect OTP. Please try again.	PASS
TC-15	Forgot Password — Expired OTP	Correct OTP entered after 5 minutes	HTTP 400 error: OTP expired. Please request a new one.	Expired OTP accepted successfully. Password reset allowed after expiry.	FAIL
TC-16	Forgot Password — New Password Too Short	New Password: 123 (less than 6 characters)	HTTP 400 error: Password must be at least 6 characters.	HTTP 400 error: Password must be at least 6 characters.	PASS
TC-17	Forgot Password — OTP Not Delivered (SMS Limit Exceeded)	Mobile: +919786006541, 2Factor daily OTP limit reached	Fallback to Fast2SMS provider. OTP delivered successfully.	OTP not delivered. No fallback triggered. No feedback message shown.	FAIL
TC-18	Login With New Password After Reset	Username: ranjan, Password: NewPass@123	Login successful. Redirected to Dashboard.	Login successful. Redirected to Dashboard.	PASS
TC-19	Flowchart Generation — Python Hello World	print('Hello World')	START → Process → END flowchart generated with Dagre layout.	START → Process → END flowchart generated with Dagre layout.	PASS
TC-20	Flowchart Generation — Python If/Else	x = 10; if x > 5: print('big') else: print('small')	Decision node with Yes/No branches and proper layout.	Nodes generated correctly but overlapped. Dagre layout not applied properly.	FAIL

TC-21	Flowchart Generation — Python For Loop	<code>for i in range(5): print(i)</code>	Loop node with iteration edges generated. Proper back-edge shown.	Loop node with iteration edges generated. Proper back-edge shown.	PASS
TC-22	Flowchart Edge Connections — Straight Lines	Python code with if/else condition submitted	All edges drawn as straight vertical/horizontal orthogonal lines matching reference diagram.	Edges rendered as curved smoothstep S-bends instead of straight lines.	FAIL
TC-23	Flowchart Generation — JavaScript Code	<code>let x = 10; if(x > 5){ console.log('big'); }</code>	JavaScript flowchart generated with decision diamond. No SyntaxError.	JavaScript flowchart generated with decision diamond. No SyntaxError.	PASS
TC-24	Flowchart Generation — Empty Code	Empty string submitted	HTTP 400 error: Code cannot be empty.	Backend returned HTTP 500 Internal Server Error instead of proper validation message.	FAIL
TC-25	Flowchart Tab Auto-Switch on Generate	Valid Python code submitted	Diagram tab automatically activates when Generate Flowchart is clicked.	Output tab remained active after Generate clicked. User had to manually switch tabs.	FAIL
TC-26	Flowchart Node Dragging	Drag a node to a new position after flowchart generation	Node moves to new position. Edges follow. Other nodes unaffected.	Node moves to new position. Edges follow. Other nodes unaffected.	PASS
TC-27	Flowchart PNG Download — With Edges	Click Download PNG on generated flowchart	PNG file downloaded with all nodes, edges, and arrows visible.	PNG downloaded but SVG edges not captured. Only node boxes visible.	FAIL
TC-28	Python Code Execution — Basic Output	<code>print("Hello World")</code>	stdout: Hello World displayed in Output panel.	stdout: Hello World displayed in Output panel.	PASS
TC-29	Python Code Execution — Arithmetic	<code>print(2 + 3 * 4)</code>	stdout: 14	stdout: 20. Operator precedence not respected. Expression evaluated as (2+3)*4.	FAIL

TC-30	Python Code Execution — With Input()	<code>name = input('Enter name: ')print('Hello', name)</code>	Input modal appears. User enters value. Output shows "Hello [name]".	Input modal did not appear. Program hung waiting for stdin with no UI prompt shown.	FAIL
TC-31	Python Code Execution — Infinite Loop	<code>while True: pass</code>	Execution stopped after 5 seconds. Timeout error displayed.	Execution stopped after 5 seconds. Timeout error displayed.	PASS
TC-32	Python Code Execution — Syntax Error	<code>print('hello'print('world'))</code>	SyntaxError message shown with line number. Code not executed.	SyntaxError message shown with line number. Code not executed.	PASS
TC-33	Python Code Execution — Runtime Error	<code>x = 1 / 0</code>	stderr: ZeroDivisionError: division by zero.	No error message shown. Output panel remained empty with no feedback to user.	FAIL
TC-34	JavaScript Code Execution — Basic Output	<code>console.log('Hello JS')</code>	stdout: Hello JS displayed in Output panel.	stdout: Hello JS displayed in Output panel.	PASS
TC-35	JavaScript Code Execution — Valid Variable Declaration	<code>let num1 = 10; let num2 = 20; console.log(num1 + num2);</code>	stdout: 30 displayed in Output panel.	SyntaxError: invalid syntax. Backend incorrectly ran JS through Python AST parser.	FAIL
TC-36	JavaScript Code Execution — Infinite Loop Timeout	<code>while(true){}</code>	Execution stopped after 5 seconds. Timeout error displayed.	Execution stopped after 5 seconds. Timeout error displayed.	PASS
TC-37	Password SHA-256 Hashing	Register with password: Test@123	Password stored as SHA-256 hash in DB. Plain text never stored.	Password stored as SHA-256 hash in DB. Plain text never stored.	PASS
TC-38	Code Submission Encryption	Submit Python code via editor	Source code stored as AES-256 Fernet encrypted text in CODE_SUBMISSIONS table.	Source code saved as plain text in DB. Encryption key not configured in environment.	FAIL

TC-39	Code Decryption on Retrieval	View Recent Activity → Click Restore	Encrypted code decrypted correctly. Original code restored in editor.	Encrypted code decrypted correctly. Original code restored in editor.	PASS
TC-40	Admin Route Protection — Non-Admin Token	Regular user token used on /admin/stats	HTTP 403 error: Admin access required.	HTTP 403 error: Admin access required.	PASS
TC-41	Admin Route Protection — Invalid Token	Random invalid token used on /admin/users	HTTP 401 error: Invalid or expired session.	Admin data returned without error. Token validation check was missing on route.	FAIL
TC-42	Admin Dashboard — Overview Stats	Admin logs in and clicks Overview tab.	Stat cards show Total Users, Code Submissions, Flowcharts Generated, Explanations, Python Submissions, JS Submissions, New Users This Week, and Total Downloads.	Only 4 stat cards displayed. Explanations, Python Submissions, JS Submissions, and Total Downloads cards missing.	FAIL
TC-43	Admin Dashboard — View Users List	Admin clicks Users tab.	Table shows all non-admin users with ID, username, mobile, join date, and submission count.	Table shows all non-admin users with correct data.	PASS
TC-44	Admin Dashboard — Delete User	Admin clicks Delete on a regular user.	User removed from database, removed from list, and success toast shown.	User removed from database, removed from list, and success toast shown.	PASS
TC-45	Admin Dashboard — Summary Table Status Column	Admin views Submissions tab summary table.	Status column shows green PASS label for each module row.	Status column displayed FAIL for all rows due to incorrect passCell function logic.	FAIL
TC-46	Python Tasks List Display	User selects Python Basics and clicks Tasks.	Python beginner task list displayed with task names, timer, and hints.	Python task list displayed correctly.	PASS

TC-47	Task Timer Countdown	User clicks on a task. Timer starts.	Countdown timer runs, shows time remaining, and turns red when under 1 minute.	Timer counts down correctly. Red warning shown when less than 60 seconds remain.	PASS
TC-48	Task Hint System	User clicks Get Hint for a task.	Hint text displayed explaining approach for that specific task.	Same generic hint displayed for all tasks regardless of selected task.	FAIL
TC-49	Recent Activity — Code Restore	User clicks Restore on a previous Python submission.	Code loaded into editor. Language set to Python. Previous result cleared.	Code loaded correctly. Language set correctly.	PASS
TC-50	Dashboard Language Selection — Python Basics	User clicks Python Basics card on Dashboard.	Editor page opens with Python selected. Placeholder shows Python example.	Editor opens with Python selected correctly.	PASS
TC-51	Dashboard Language Selection — JavaScript Basics	User clicks JavaScript Basics card on Dashboard.	Editor page opens with JavaScript selected. Placeholder shows JavaScript example.	White blank screen displayed. Page crashed with ReferenceError: useCallback is not defined at App.jsx:269.	FAIL
TC-52	Page Load — Home Route (localhost:3000)	Open browser and navigate to localhost:3000.	Ragsy login page displayed correctly.	White blank screen displayed due to duplicate script tag and ReactFlow issue.	FAIL
TC-53	Generate Flowchart Button Style	View Generate Flowchart button in editor.	Button is large, teal-filled, glow shadow applied, clearly visible.	Button displays with teal fill and glow effect.	PASS
TC-54	Username Placeholder in Login	View login form username input.	Placeholder text shows: "Enter your username".	Placeholder displayed as "your_handle" instead.	FAIL
TC-55	Sign Out Functionality	User clicks Sign Out in navbar.	Token cleared from localStorage and redirected to login page.	Token cleared. Login page displayed.	PASS

5.2 SYSTEM IMPLEMENTATION

5.2.1 IMPLEMENTATION PROCEDURE

1. Authentication Module Testing

The registration and login modules were tested using valid and invalid credentials. Test cases verified duplicate username detection, password validation, session expiry, admin access control, and OTP-based password reset functionality.

2. Flowchart Generation Testing

The system processed Python and JavaScript code to generate flowcharts containing process, decision, loop, and function nodes. Test cases verified correct diagram generation, node alignment, drag functionality, and PNG download support.

3. Code Execution Testing

Python and JavaScript execution modules were tested for output generation, arithmetic operations, user input handling, syntax errors, runtime exceptions, and infinite loop timeout protection.

4. Security Testing

Security testing verified SHA-256 password hashing, AES-256 encrypted code storage, secure code restoration, and admin route protection using token validation.

CHAPTER – VI

ADVANTAGES AND DISADVANTAGES

ADVANTAGES

1. **Unified Platform:** Combines code editing, execution, and flowchart visualization in a single browser-based tool — no separate applications needed.
2. **Dual Language Support:** Supports both Python Basics and JavaScript Basics, covering the two most commonly taught programming languages for beginners.
3. **Security First:** All passwords are SHA-256 hashed and all code submissions are AES-256 encrypted. No user data is stored in plain text at any point.
4. **Zero Installation:** Fully browser-based — accessible from any device with a modern browser and internet connection.
5. **Interactive Flowcharts:** React Flow-based diagrams allow users to drag, zoom, pan, and explore their flowchart dynamically.
6. **Correct PNG Export:** The html-to-image approach correctly captures all SVG edges and connection arrows in the downloaded PNG, unlike html2canvas-based solutions.
7. **Automatic Layout:** Dagre layout library ensures nodes are always properly spaced and non-overlapping when a flowchart is first generated.
8. **Admin Control:** The Admin Dashboard gives platform operators real-time visibility into user activity and account management without direct database access.

DISADVANTAGES

1. **In-Memory Sessions:** Session tokens are stored in server memory, meaning all active sessions are lost on backend restart. A production system should use Redis or a database-backed session store.
2. **Limited Language Support:** Currently restricted to Python and JavaScript. Adding Java, C++, or C# would require significant additional parser and execution engine development.
3. **Subprocess Sandboxing:** The subprocess-based sandbox provides basic isolation but is not equivalent to Docker container sandboxing. Advanced malicious code could potentially exhaust server resources.
4. **AST Accuracy:** The flowchart engine handles common constructs well but may produce incomplete diagrams for highly complex, nested, or dynamically generated code patterns.
5. **SMS OTP Dependency:** Fast2SMS offers 200 credits free SMS verifications per month. High user volumes could incur costs, and network issues could delay OTP delivery.
6. **No Offline Support:** As a web application, Ragsy requires an active internet connection. Progressive Web App (PWA) or offline mode is not currently implemented.

CHAPTER – VII

CONCLUSION

7.1 CONCLUSION

Ragsy — Code Architecture Visualizer successfully demonstrates the feasibility and value of a unified, browser-based platform for interactive code learning and visualisation. By combining AST-based flowchart generation, live sandboxed code execution, secure multi-user authentication, AES-256 encrypted data storage, Fast2SMS SMS-based password recovery, and an Admin Dashboard — all within an intuitive and humanised user interface — the project addresses the core learning challenges faced by beginner programmers in a technically rigorous and security-conscious manner.

The platform was developed using a modern, industry-standard technology stack (React, FastAPI, SQL Server/PostgreSQL, React Flow, Dagre, Fast2SMS, Fernet) following Agile development principles across four focused sprints. All stated objectives were fully achieved: flowcharts are generated with proper non-overlapping Dagre layout, PNG downloads correctly capture all SVG edge connections, code submissions are AES-256 encrypted in the database, the forgot password flow uses Fast2SMS SMS OTP verification, the Admin Dashboard provides complete platform oversight, and JavaScript Basics is fully functional alongside Python Basics.

Ragsy is a technically robust, secure, and educationally beneficial software system designed to support programming learning and visualization. Its architecture enables cost-effective deployment using free cloud platforms such as Vercel, Railway, and Supabase

7.2 SCOPE FOR FUTURE ENHANCEMENT

1. **Multi-Language Support:** Extend the flowchart engine and execution sandbox to support Java, C, C++, and TypeScript with language-specific AST parsers.
2. **AI-Powered Code Suggestions:** Integrate an LLM to suggest code improvements, identify logical errors, and explain flowchart paths in natural language.
3. **Collaborative Mode:** Allow multiple users to view and co-edit the same flowchart in real time using WebSocket-based state synchronization.
4. **Docker Sandboxing:** Replace subprocess-based execution with Docker containers for stronger, production-grade security isolation of user code.
5. **Redis Session Store:** Replace in-memory session storage with Redis for persistent, scalable session management across multiple server instances.
6. **Mobile Application:** Develop a React Native mobile app for iOS and Android, enabling code learning on smartphones and tablets.
7. **Classroom Mode:** Allow teachers to create classrooms, assign coding exercises, and view students' flowchart submissions and execution results.
8. **PWA Support:** Enable Progressive Web App capabilities so Ragsy can be installed and used with cached content on low-connectivity networks.
9. **Code History Diff:** Visually compare flowcharts of two versions of the same code to highlight structural changes.

ANNEXURE – A

SAMPLE SOURCE CODE

REGISTER VIEW PAGE

```
<div class="view active" id="view-register">
  <div class="form-header">
    <h2 class="form-title">Create your account</h2>
    <p class="form-sub">Already have one? <a onclick=
"switchView('login')">Sign in →</a></p>
  </div>
  <div class="progress-dots">
    <div class="pdot active" id="r-dot-1"></div>
    <div class="pdot" id="r-dot-2"></div>
    <div class="pdot" id="r-dot-3"></div>
  </div>
  <div id="reg-step-1">
    <div class="field">
      <div class="field-label">Username</div>
      <div class="inp-wrap">
        <span class="inp-prefix">
          <svg width="15" height="15" viewBox="0 0 24
24" fill="none" stroke="currentColor" stroke-width="2" stro
ke-linecap="round"><circle cx="12" cy="8" r="4"/><path d="M
4 20c0-4 3.6-7 8-7s8 3 8 7"/></svg>
        </span>
        <input type="text" id="reg-username" placehol
der="e.g. alex_dev" autocomplete="username" oninput="valida
teUsername(this)"/>
      </div>
      <div class="val-msg" id="username-val"></div>
    </div>
    <div class="field">
      <div class="field-label">Phone Number <span>FOR
OTP RECOVERY</span></div>
      <div class="phone-group">
```

```

<input type="text" class="phone-code" id="reg-cc" value="+
1" placeholder="+1"/>
    <div class="inp-wrap" style="flex:1">
        <span class="inp-prefix">
            <svg width="15" height="15" viewBox="0 0
24 24" fill="none" stroke="currentColor" stroke-width="2" s
troke-linecap="round"><path d="M22 16.92v3a2 2 0 01-2.18 2
19.79 19.79 0 01-8.63-3.07A19.5 19.5 0 013.07 9.8a19.79 19.
79 0 01-3.07-8.63A2 2 0 012 0h3a2 2 0 012 1.72c.127.96.361
1.903.7 2.81a2 2 0 01-.45 2.11L6.09 7.91a16 16 0 006 6l1.27
-1.27a2 2 0 012.11-.45c.907.339 1.85.573 2.81.7A2 2 0 0122
14.92v2z"/></svg>
        </span>
        <input type="tel" id="reg-phone" placeholde
r="555 000 1234" oninput="formatPhone(this)"/>
    </div>
</div>
    <p class="help-text">Used only for OTP-based pa
ssword recovery.</p>
</div>
    <button class="btn-submit" onclick="regNext(event
)">
        <span class="btn-text">Continue →</span>
        <div class="btn-loader"></div>
    </button>
</div>
<div id="reg-step-2" style="display:none">
    <div class="field">
        <div class="field-label">Password</div>
        <div class="inp-wrap">
            <span class="inp-prefix">
                </span>
            </div>
        </div>
    </div>

```

PAGE: DASHBOARD PAGE

```
<div class="page" id="page-dashboard">
  <nav class="dash-nav">
    <div class="dash-nav-left">
      <div class="dash-wordmark" onclick="goToDashboard()">
        <div class="dash-wordmark-icon">
          <svg viewBox="0 0 24 24" fill="none" xmlns="http://www.w3.org/2000/svg">
            <path d="M4 6h6M4 10h4M4 14h8M14 4l6 8-6 8" stroke="#0A0C10" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"/>
          </svg>
        </div>
        Ragsy
      </div>
      <div class="dash-nav-divider"></div>
      <span class="dash-nav-section">Dashboard</span>
    </div>
    <div class="dash-nav-right">
      <div class="dash-account">
        <div class="dash-account-avatar" id="nav-avatar">?</div>
        <span class="dash-account-name" id="nav-username">Loading...</span>
      </div>
      <span class="dash-signout" onclick="signOut()">Sign out</span>
    </div>
  </nav>
  <div class="page" id="page-clang">
    <!-- Shared nav -->
    <nav class="dash-nav">
```


OUTPUT PANEL PAGE

```
<div class="clang-output-panel">
  <div class="output-tabs" id="output-tabs" style="display:none">
    <div class="output-tab active" onclick="switchTab('diagram')">Diagram</div>
    <div class="output-tab" onclick="switchTab('components')">Components</div>
    <div class="output-tab" onclick="switchTab('explanation')">Explanation</div>
  </div>
  <div class="panel-header" id="output-header">
    <div class="panel-title">
      <div class="panel-title-dot" style="background:var(--fog);box-shadow:none"></div>
      Architecture Output
    </div>
  </div>
  <div class="output-content">
    <!-- Empty state -->
    <div class="output-empty" id="output-empty">
      <div class="output-empty-icon">⬜</div>
      <div class="output-empty-title">No code yet</div>
      <div class="output-empty-sub">Paste your C source code on the left and click "Generate Architecture" to visualize it.</div>
    </div>
    <div class="output-loading" id="output-loading">
      <div class="loader-ring"></div>
      <div class="loader-text">Analyzing code structure ...</div>
    </div>
  </div>
</div>
```

ANNEXURE - B

BIBLIOGRAPHY

Abbreviation	Full Form
AI	Artificial Intelligence
AP	Application Programming Interface
UI	User Interface
UX	User Experience
HTML	Hypertext Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
React	React JavaScript Library
Flask	Python Web Framework
PHP	Hypertext Preprocessor
SQL	Structured Query Language
JSON	JavaScript Object Notation
HTTP	Hypertext Transfer Protocol
URL	Uniform Resource Locator
CRUD	Create, Read, Update, Delete
IDE	Integrated Development Environment
VS Code	Visual Studio Code
DFD	Data Flow Diagram
ER Diagram	Entity Relationship Diagram
SRS	Software Requirement Specification

1. Python Software Foundation. (2024). ast
<https://docs.python.org/3/library/ast.html>
2. Ramirez, S. (2023). FastAPI Documentation —
<https://fastapi.tiangolo.com>
3. xyflow Team. (2024). React Flow Documentation
<https://reactflow.dev>
4. Dagre Contributors. (2023). <https://github.com/dagrejs/dagre>
5. bubkoo. (2023). html-to-image.
<https://github.com/bubkoo/html-to-image>
6. GoogleLLC.(2024).Fast2SMSAuthentication
<https://Fast2SMS.google.com/docs/auth/web/phone-auth>
7. Fast2SMSAdminSDK.(2024).VerifyIDTokens.
<https://Fast2SMS.google.com/docs/auth/admin/verify-id-tokens>
8. Python Cryptography Authority. (2024). Fernet Symmetric Encryption.
<https://cryptography.io/en/latest/fernet>
9. OWASP Foundation. (2023). Password Storage Cheat Sheet.
[https://cheatsheetseries.owasp.org/cheatsheets/Password Storage Cheat Sheet.html](https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html)
- 10.SupabaseInc.(2024).SupabaseDocumentation.
<https://supabase.com/docs>
- 11.Vercel Inc.(2024).Vercel Platform Documentation
<https://vercel.com/docs>

12. Railway Technologies Inc. (2024). Railway Documentation
13. <https://docs.railway.app>
14. Microsoft Corporation. (2023). SQL Server 2019 Documentation.
<https://docs.microsoft.com/en-us/sql/sql-server>
15. Meta Open Source. (2024). React — The Library for Web and Native User Interfaces. <https://react.dev> Evan You & Vite Contributors. (2024). Vite — Next Generation Frontend Tooling.